

BUGBOARD 2026

SCELTE DI PROGETTAZIONE E IMPLEMENTAZIONE

PROBLEMA E DOMINIO

Il **problema**: tracciare le segnalazioni software di un'organizzazione con stati, responsabilità e scadenze espliciti

Issue di 4 tipi: **bug, feature, question, documentation**

3 ruoli con permessi distinti:

Admin —
gestisce utenti,
assegna, chiude

User — crea,
risolve,
commenta

Readonly —
consulta i soli
bug

Ciclo di vita esplicito: **TODO**
→ **IN_PROGRESS** → **DONE** /
EXPIRED / **CLOSED**

Ogni assegnazione genera una **notifica** al destinatario

Rocco Molinari - N86005071
Daniele Megna - N86005120

STACK TECNOLOGICO

Java 17 + Spring Boot 3.2.5 — tipizzazione forte; IoC/DI, convention over configuration, Tomcat embedded

PostgreSQL — ACID per l'integrità; ibrido: array text[] per le etichette, jsonb per i commenti

Spring Data JPA / Hibernate — astrazione dal dialetto SQL; query parametrizzate → SQL injection neutralizzata

Vue.js 3 — reattività (Virtual DOM); escaping automatico nei template → anti-XSS

Vite · Axios · Tailwind — dev server ESM e tree-shaking; AJAX; UI responsiva senza CSS monolitici

Nginx — asset statici + reverse proxy: same-origin, fallback SPA, limiti upload

GCP Compute Engine — VPC per l'isolamento di rete, IP statico, firewall

PROCESSO

IBRIDO: PIANIFICAZIONE A MONTE, COSTRUZIONE ITERATIVA

Rocco Molinari - N86005071
Daniele Megna - N86005120

PROCESSO DI SVILUPPO

Ibrido guidato dal profilo di rischio:

analisi plan-driven a monte — requisiti noti e stabili a priori → specifica completa up-front

costruzione iterativa e incrementale (ispirata a Scrum) — il rischio era nella realizzazione: tecnologie al primo impiego, integrazione FE/BE

Pratiche adottate: backlog prioritizzato · incrementi integrati e testati · allineamenti frequenti · review e retrospettive informali

Scartato consapevolmente: ruoli formali e cerimonie timeboxed — overhead sproporzionato per un team di 2

Evidenza empirica (Git): 87 commit apr–giu 2026 — setup → picco architetturale (maggio) → stabilizzazione e bug-fixing (giugno)

Rocco Molinari - N86005071
Daniele Megna - N86005120

REQUISITI

NOTI A PRIORI → ANALISI UP-FRONT COMPLETA

Rocco Molinari - N86005071
Daniele Megna - N86005120

REQUISITI

Analisi up-front: use case UML + 2 casi d'uso dettagliati (modello Cockburn) con mock-up

Funzionali (RF-01...07): autenticazione · gestione issue · filtri · assegnazione e notifiche · etichette · ruolo readonly · scadenze

Non funzionali misurabili e verificabili:

Vincoli progettuali (RS-01...09): stack imposto · API REST · layering · versioning ottimistico · anti-XSS / anti-SQL injection

Sicurezza — password mai in chiaro (BCrypt), autorizzazione per ruolo

Integrità — niente lost update su modifiche concorrenti → errore esplicito

Affidabilità — disponibilità $\geq 99.5\%$ · RTO 15 min · RPO 24 h · self-healing

Prestazioni — $p95 \leq 1$ s con 20 utenti concorrenti (misurato: ≈ 165 ms)

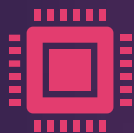
Rocco Molinari - N86005071
Daniele Megna - N86005120

ARCHITETTURA & DESIGN

OGNI SCELTA MOTIVATA DA UN REQUISITO

Rocco Molinari - N86005071
Daniele Megna - N86005120

VISTA D'INSIEME



Three Layering: SPA (Vue) ⇔ API REST ⇔ Spring Boot ⇔ PostgreSQL



Contratto REST stabile → FE e BE indipendenti: sviluppo parallelo (il FE mocka le API), sostituibilità



Topologia a reverse proxy: il browser parla solo con Nginx; /api/ instradato sulla rete privata
BE e DB **mai esposti su internet** (VPC, defense in depth)



SPA: aggiornamenti senza reload → reattività; complessità spostata sul client

Rocco Molinari - N86005071
Daniele Megna - N86005120

LAYERING BACKEND



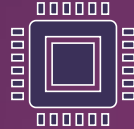
4 livelli, dipendenze solo verso il basso:

Controller — presentation:
HTTP/JSON, estrae la sessione

Service — business: regole,
autorizzazione per ruolo

Repository — data access:
Spring Data JPA

Model — dominio: entità JPA

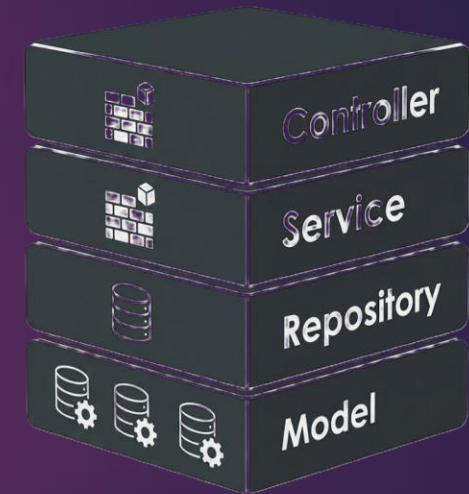


Trasversali:

errori centralizzati — gerarchia
ApiException → status HTTP, un
solo @RestControllerAdvice



Data layer ottimizzato:
proiezione diretta su
DTO (mai il bytea in
lista) · polling
incrementale su version



Rocco Molinari - N86005071
Daniele Megna - N86005120

MODELLO DI DOMINIO



Entità: Utente (ruolo) ·
Issue · Sessione ·
Notifica —
Commento —
Commento
incorporato nella
Issue



Issue è il nucleo:
stato, tipo, priorità,
scadenza, etichette,
allegato, 3 relazioni
verso Utente



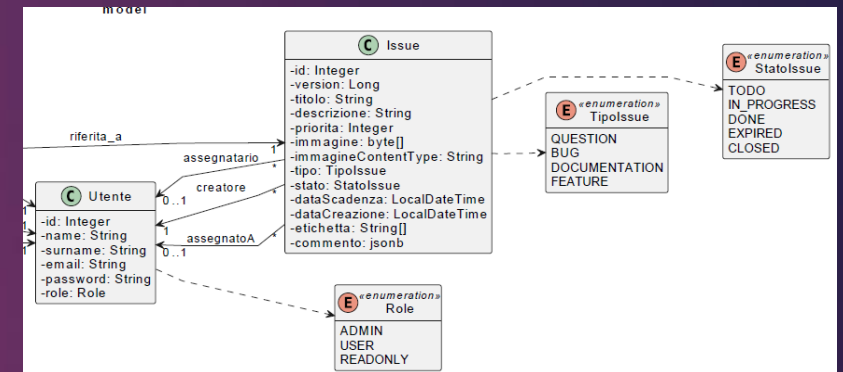
**Scelte di persistenza
motivate:**

campo **version (@Version)**
→ concorrenza ottimistica
(RS-06)

etichette **text[]**, commenti
jsonb → PostgreSQL ibrido
relazionale/documentale,
niente tabelle di join per
dati consultivi

sessione con chiave **UUID**
→ previene ID-
enumeration

password **hash BCrypt**, mai
in chiaro (RNF-01)



Rocco Molinari - N86005071
Daniele Megna - N86005120

DESIGN PATTERN & SOLID PRINCIPLES



Builder — IssueBuilder:
conversioni
DTO→dominio
accentrate; nessuna
entità inconsistente
(l'unica via è build())



Observer / event-driven
— IssueAssegnataEvent
+ NotificaListener: le
notifiche escono da
IssueService → SRP, zero
accoppiamento verso
NotificaRepository



DTO + Mapper —
payload profilati per
ruolo e caso d'uso; il
modello interno non
transita mai sulla rete



Singleton —
implementato
mediante Dependency
Injection delle istanze



SOLID PRINCIPLES —
attentamente implementati
nel backend

Rocco Molinari - N86005071
Daniele Megna - N86005120

API REST

Risorse sotto /api/ — auth · issues · user · notifies, verbi HTTP semanticamente corretti (GET/POST/PUT/DELETE)

Sessione nell'header X-Session-Id, mai nell'URL → non compare in log del server e cronologia del browser

Errori uniformi: ApiException → 400 / 403 / 404 / 409, tradotti in un unico punto (GlobalExceptionHandler)

- conflitto da lock ottimistico → **409 Conflict**: "ricarica e riprova"

Risposte profilate per ruolo: IssueResponse · IssueResponseUser · IssueSpecificAdmin — ogni client vede solo ciò che gli spetta

Endpoint delta: GET /api/issues/versions restituisce le sole coppie (id, version) → dashboard aggiornata a costo costante

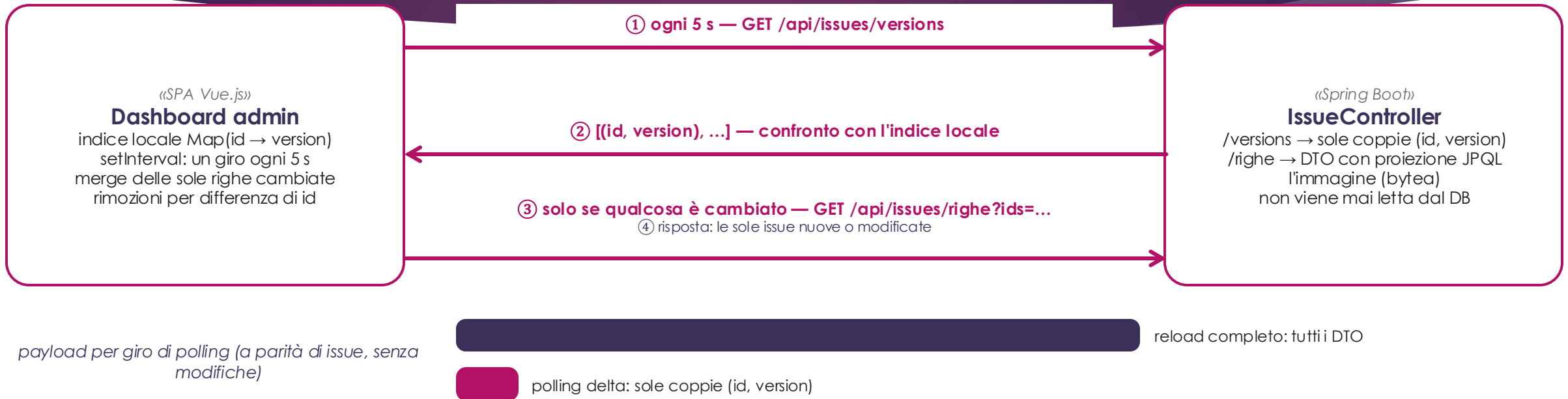
LOGIN E SESSIONE



Dopo il login: ogni richiesta è autenticata

l'interceptor Axios inietta X-Session-Id su ogni chiamata · il sid non transita mai nell'URL (fuori da log e cronologia)
il Controller estrae l'header, il Service risolve sid → utente e ne verifica il ruolo · sessione stateful: revoca immediata

DASHBOARD ADMIN — POLLING DELTA



Perché il polling delta?

riusa il campo @Version introdotto per la concorrenza ottimistica (RS-06): un meccanismo, due funzioni
costo per giro ~costante al crescere delle issue (RNF-04) · trade-off: aggiornamento differito fino a 5 s, zero push

NOTIFICHE LATO USER



Perché in due tempi?

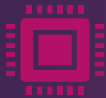
generazione disaccoppiata (Observer, RF-04): chi assegna non sa chi notifica → SRP, dipendenze zero
consegna a costo minimo: un intero ogni 5 s · liste solo su variazione o richiesta · latenza ≤ 5 s accettata (niente push)

QUALITÀ & TESTING

CASI DI TEST DERIVATI SISTEMATICAMENTE

Rocco Molinari - N86005071
Daniele Megna - N86005120

STRATEGIA DI TESTING



Livello: unit test sui Service — la logica di business verificata in isolamento



JUnit 5 + Mockito: repository e collaboratori sostituiti da mock → test rapidi, deterministici, senza DB



Due strategie complementari:

black-box (funzionale) — casi derivati dalla specifica: classi di equivalenza

white-box (strutturale) — casi derivati dal codice: copertura delle condizioni



Sotto test 3 metodi: assignIssueToUser (5 parametri) · aggiungiComment o (4) · deleteUser (2)



Test plan documentato: 39 test totali, tutti verdi, condition coverage completa

Rocco Molinari - N86005071
Daniele Megna - N86005120

TESTING BLACKBOX

Il dominio di **ogni parametro** è partizionato in **classi di equivalenza** valide e non valide

Criterio di combinazione each-choice: ogni classe compare in almeno un test, 9 test totali

- vs all-combinations: oltre 100 combinazioni per assignIssueToUser, in gran parte ridondanti

Esempio — assignIssueToUser, 5 parametri / 13 classi: issueId {esiste, no} · userEmail {USER, inesistente, non-USER} · expiringDate {null, valorizzata} · expectedVersion {null, uguale, diversa} · adminSID {admin, nulla, non-admin}

Limite dichiarato — masking: combinando più classi non valide, la prima eccezione maschera le altre

- documentato caso per caso e **compensato dal white-box**

TESTING WHITEBOX

Criterio adottato: condition coverage completa — ogni condizione atomica valutata a vero e a falso almeno una volta

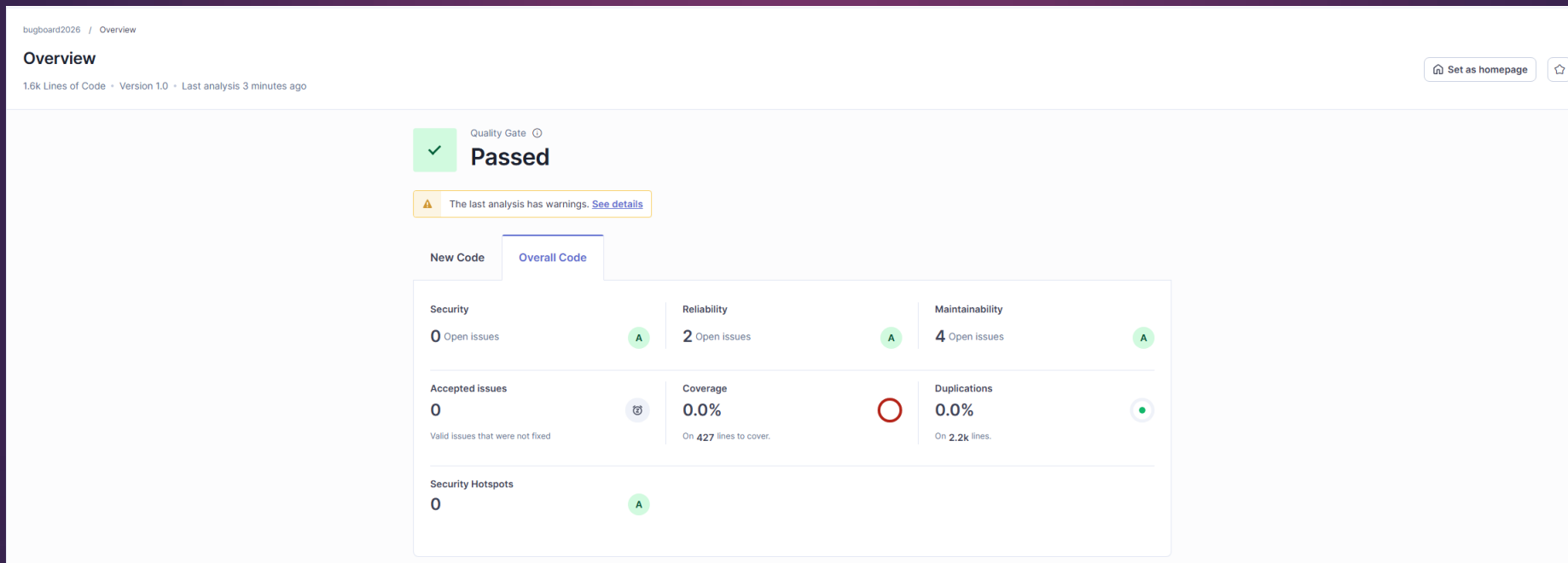
- implica la copertura di istruzioni e rami (criteri più deboli)

Tracciabilità totale: per ogni condizione, la tabella indica il test che la rende vera e quello che la rende falsa

30 test strutturali: 12 su assignIssueToUser · 13 su aggiungiCommento · 5 su deleteUser

Copre ciò che il black-box non raggiunge: lo stato interno degli oggetti (stato della issue, ownership) e le classi mascherate (expectedVersion)

REPORT SONARQUBE



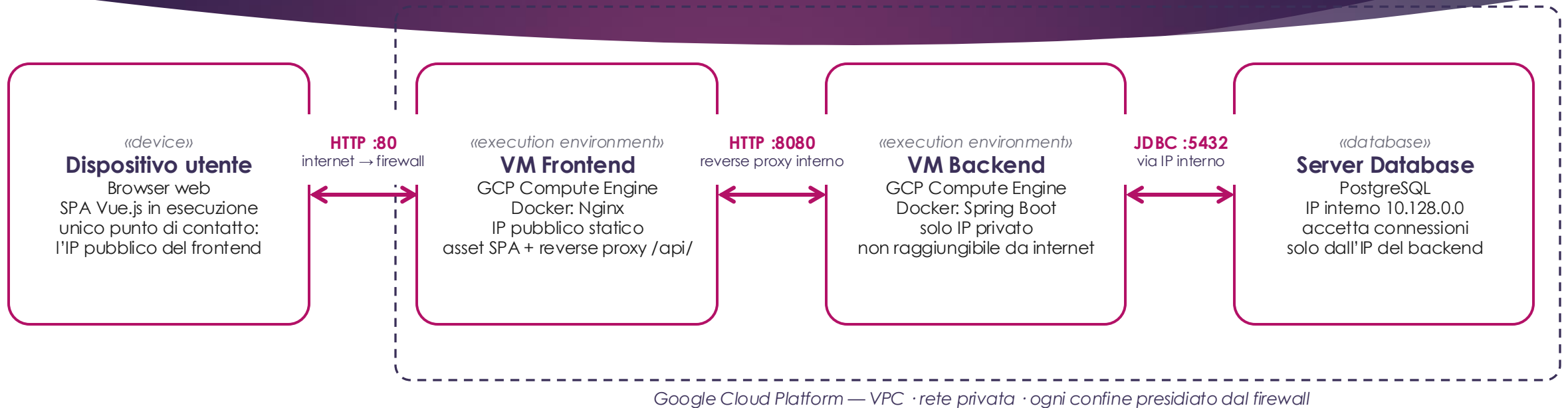
Rocco Molinari - N86005071
Daniele Megna - N86005120

DEPLOYMENT & CONFIG

RIPRODUCIBILE, ISOLATO, AUTOMATIZZATO

Rocco Molinari - N86005071
Daniele Megna - N86005120

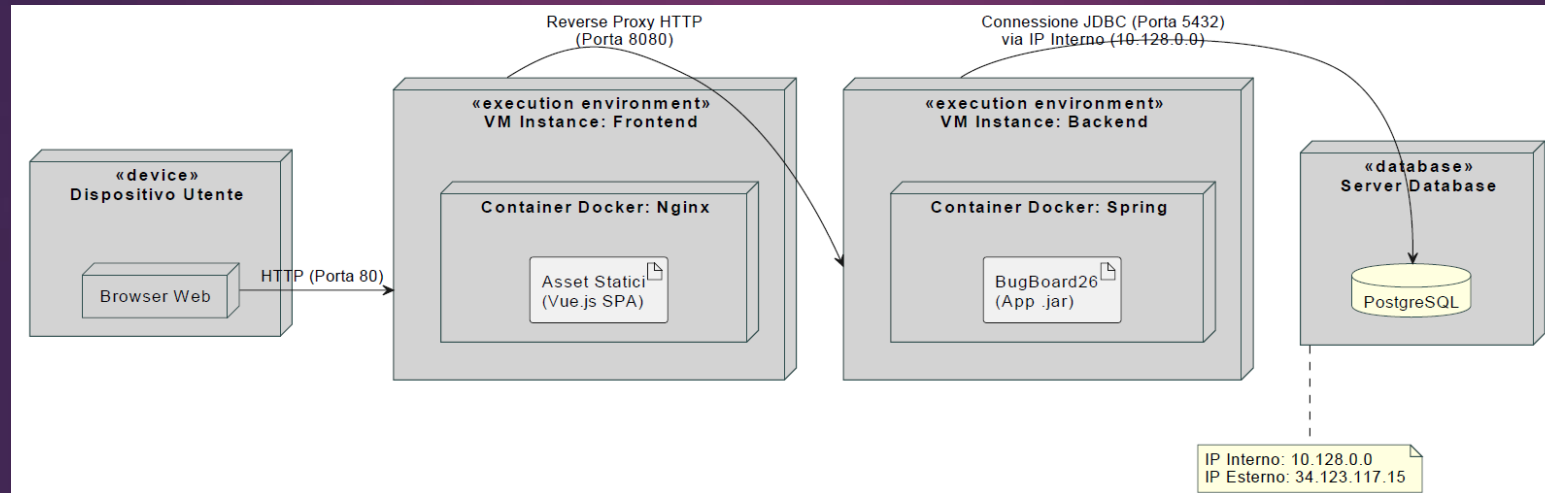
ARCHITETTURA DI SISTEMA



Perché tre nodi separati?

isolamento dei guasti · scaling indipendente per profilo di carico (asset statici / CPU-bound / I/O-bound)
defense in depth: solo il frontend è esposto · rilasci indipendenti (due job di deploy)

DEPLOYMENT DIAGRAM



Rocco Molinari - N86005071
Daniele Megna - N86005120

CONTAINERIZZAZIONE



Dockerfile multi-stage
per FE e BE: build
separata dal runtime
minimale (JRE · Nginx
Alpine)

immagini leggere, superficie
d'attacco ridotta



**Docker Compose in
locale:** stessa topologia
della produzione →
parità di ambienti



**Immagini immutabili su
GHCR:** un nodo guasto
si ricrea dall'immagine
→ RTO 15 min (RNF-03)



restart: unless-stopped
→ self-healing senza
intervento manuale

Rocco Molinari - N86005071
Daniele Megna - N86005120

AUTOMATIZZAZIONE DEPLOYMENT



CI/CD con GitHub Actions, trigger su push / pull request verso main



Pipeline: test unitari + build → immagini Docker → push su GHCR → deploy SSH parallelo sui due nodi (solo push su main)



Segreti fuori dal repository: chiave SSH nei GitHub Secrets



3 nodi su GCP: FE pubblico (Nginx) · BE su solo IP privato in VPC · DB che accetta solo il BE

isolamento dei guasti · scaling indipendente · defense in depth · rilasci indipendenti



Rocco Molinari - N86005071
Daniele Megna - N86005120

CHIUSURA

COSA DIFENDIAMO E COSA RIFAREMMO

Rocco Molinari - N86005071
Daniele Megna - N86005120

CONFIGURATION MANAGEMENT

Git + GitHub (repo privato)
come unica fonte: codice,
storia, CI

98 commit (apr-lug 2026),
bilanciati 52 + **46** → conoscenza
condivisa del codice

**Code frequency = evidenza
empirica** dell'approccio
iterativo dichiarato

RETROSPETTIVA E SINTESI

► Le decisioni che difendiamo:

- processo ibrido guidato dal profilo di rischio
- topologia a 3 nodi
- concorrenza ottimistica end-to-end: @Version → 409 → retry lato client (RNF-02)
- testing sistematico: classi di equivalenza + condition coverage

► Cosa ha funzionato: il contratto REST stabile ha reso il lavoro davvero parallelo in un team di 2

► Trade-off tecnici riconosciuti:

- polling delta (5 s): costo costante, zero push — ma aggiornamento mai istantaneo
- reverse proxy: zero CORS, BE mai esposto — ma l'hop domina la latenza misurata
- ripristino (RTO 15 min) al posto della ridondanza: né repliche né load balancer
- stato tutto nel DB: nodi effimeri — ma persistenza su un solo nodo (RPO 24 h)